SOFTWARE REQUIREMENTS SPECIFICATION

MedConnect : Dental Core

Clinical Registration & Telehealth Access Platform

Document Reference	SRS-MC-DC-2025-001
Version	1.0.0 Initial Release
Prepared By	Sarah Abumandil
Date	July 15, 2025
Standard	IEEE 830 / ISO/IEC/IEEE 29148:2018
Classification	Academic , Prototype Specification
Status	FINAL

This document constitutes the authoritative specification for the MedConnect Dental Coresystem and supersedes all prior prototype-phase assumptions and wireframes.

Revision History

Version	Date	Author	Description
1.0.0	July 15, 2025	Sarah Abumandil	Initial SRS derived from MedConnect prototype codebase

Table of Contents

1 Introduction4
1.1 Purpose4
1.2 Scope4
1.3 Definitions, Acronyms & Abbreviations4
1.4 References5
1.5 Document Overview5
2 Overall Description5
2.1 Product Perspective5
2.2 Product Functions6
2.3 User Classes & Characteristics6
2.4 Operating Environment7
2.5 Design & Implementation Constraints7
2.6 Assumptions & Dependencies7
3 System Features & Functional Requirements8
3.1 Identity Access & Authentication (IAA)8
3.2 Specialist Dashboard9
3.3 Clinical Registration & Booking Form10
3.4 Appointment Confirmation & Notification11
3.5 State-Driven Navigation12
4 Data Requirements12
5 External Interface Requirements13
5.1 User Interfaces13
5.2 Software Interfaces13
6 Non-Functional Requirements13
7 Constraints & Compliance14
8 Appendix A — Use Case Summary15
9 Appendix B — State Machine Diagram16
10 Appendix C — Data Dictionary16

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional and non-functional requirements for MedConnect — Dental Core, a HIPAA-aligned web-based clinical registration and appointment booking platform for dental practices. This document serves as the authoritative reference for all development, testing, and validation activities and translates the MedConnect prototype into unambiguous engineering specifications.

1.2 Scope

MedConnect — Dental Core is a single-page web application (SPA) enabling patients to:

- Authenticate their identity via a validated intake form.
- Browse a roster of dental specialists with rich profile data.
- Complete a multi-section clinical registration and appointment booking form.
- Receive asynchronous SMS/push confirmation of their appointment.

The system is a fully client-side prototype using vanilla JavaScript modules (data.js, state.js, app.js) and is deployable on any static hosting provider (e.g., Vercel). No server-side computation or live telephony infrastructure is included in v1.0; the SMS pipeline is simulated via an asynchronous state machine.

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
SPA	Single-Page Application — all views rendered by client-side JS
HIPAA	Health Insurance Portability and Accountability Act (US)
IAA	Identity Access & Authentication subsystem
DDS / DMD	Doctor of Dental Surgery / Doctor of Medicine in Dentistry
Slot	A discrete appointment time-block offered by a dentist on a given date
Confirmation ID	Unique alphanumeric token (format MC-XXXX) assigned at booking
Toast	Transient non-blocking notification overlay with auto-dismiss
Booking Status	Finite-state enum: idle | validating | queued | sending | confirmed | failed
ISO Date	Date string in YYYY-MM-DD format per ISO 8601
DXA	Document XML Attribute unit; 1440 DXA = 1 inch (used in layout context only)

UX	User Experience
UI	User Interface

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering
- MedConnect Dental Core Source Code — index.html, app.js, state.js, data.js, styles.css (July 2025)
- US Department of Health & Human Services — HIPAA Security Rule, 45 CFR Part 164
- WCAG 2.1 Level AA — Web Content Accessibility Guidelines

1.5 Document Overview

Section 2 provides an overall system description. Section 3 specifies all functional requirements grouped by feature. Sections 4–6 address data, interfaces, and non-functional requirements. Section 7 covers constraints and compliance. Appendices provide use-case summaries, state-machine diagrams, and a data dictionary.

2. Overall Description

2.1 Product Perspective

MedConnect — Dental Core operates as a standalone SPA with no dependency on a backend API in v1.0. It is one component of a hypothetical broader MedConnect clinical platform. Figure 1 illustrates the system context:

System Layer	Component	Responsibility
Presentation	app.js (View Router)	Renders HTML views; owns DOM lifecycle
State	state.js (AppState)	Manages canonical app state; pub/sub event bus
Data	data.js (DentalDB)	Provides dentist records, slot matrices, symptom categories
Hosting	Vercel / Static CDN	Serves static assets globally; zero server-side logic
Notification	SMS Simulator (state.js)	Asynchronous booking pipeline with jitter emulation

The system communicates between layers via a subscriber pattern: state.js emits to all app.js listeners on every mutation, triggering a full re-render of the active view.

2.2 Product Functions

At the highest level, the system shall perform the following functions:

- F1 — Identity Verification: collect and validate patient identity credentials.
- F2 — Specialist Discovery: display and filter dental specialist profiles.
- F3 — Clinical Registration: multi-section intake form with real-time validation.
- F4 — Slot Selection: date/time picker from dynamically generated availability matrices.
- F5 — Booking Confirmation: async pipeline culminating in confirmation record creation.
- F6 — Notification Delivery: simulated SMS and push notification dispatch.
- F7 — State Navigation: context-aware bottom navigation with gated access.

2.3 User Classes & Characteristics

User Class	Description	Technical Proficiency
Patient (Primary)	Adult seeking dental appointment; may have no technical background	Low to moderate

Clinical Admin	Dental practice staff reviewing prototype; evaluating workflow UX	Moderate
Developer / QA	Engineer maintaining or extending the codebase	High

2.4 Operating Environment

Factor	Requirement
Browsers	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+ (latest two major versions)
Devices	Desktop (1024px+), Tablet (768px+), Mobile (375px+)
Connectivity	Initial asset load requires network; post-load operates offline (no API calls)
Hosting	Vercel static CDN or any HTTP/HTTPS web server (Apache, Nginx, GitHub Pages)
JavaScript	ES2020+ (IIFE module pattern; no bundler required in v1.0)
Fonts	Google Fonts CDN (Fraunces, Inter, IBM Plex Mono) — graceful degradation if unavailable

2.5 Design & Implementation Constraints

- CON-1: No server-side processing. All logic executes in the client browser.
- CON-2: No persistent storage. Refreshing the browser resets all state to initial values.
- CON-3: No live telephony. SMS delivery is fully simulated.
- CON-4: Script load order dependency — data.js must load before state.js, which must load before app.js.
- CON-5: Slot availability is deterministically pseudo-random; it does not query a live scheduling system.
- CON-6: HIPAA compliance is a design principle of the prototype, not a legally certified implementation.

2.6 Assumptions & Dependencies

- ASM-1: Users access the system via a modern web browser with JavaScript enabled.
- ASM-2: Phone numbers are assumed to be US-format 10-digit numbers.
- ASM-3: The date/time of slot generation is anchored to the client's local clock.
- ASM-4: Sunday availability is suppressed to simulate realistic clinic scheduling.
- DEP-1: Google Fonts CDN availability for typography (non-critical; system fonts used as fallback).

3. System Features & Functional Requirements

Each requirement follows the format: [ID] — [Priority] — [Requirement statement].
Priority: M = Must Have | S = Should Have | C = Could Have

3.1 Identity Access & Authentication (IAA)

The IAA module presents the entry screen of the application. The patient must supply valid credentials before accessing any other view.

Req ID	Pri	Requirement
FR-IAA-01	M	The system shall display a login card containing fields for: Full Legal Name, Mobile Phone, and Email (optional).
FR-IAA-02	M	The system shall validate Full Legal Name: minimum 3 characters; only letters, spaces, apostrophes, and hyphens permitted.
FR-IAA-03	M	The system shall validate Mobile Phone: must resolve to exactly 10 numeric digits (US format).
FR-IAA-04	M	The system shall auto-format the phone field in real time to (XXX) XXX-XXXX as the user types.
FR-IAA-05	S	The system shall validate Email when provided: must match the pattern [local]@[domain].[tld].
FR-IAA-06	M	On submission with validation errors, the system shall render inline error messages below each failing field and apply an input--error CSS class to the input.
FR-IAA-07	M	On successful validation, the system shall call AppState.authenticate() and transition to the DASHBOARD view.
FR-IAA-08	M	On successful authentication, the system shall display a success toast: title "Identity Verified"; body "Welcome, [FirstName]. Loading your dashboard...".
FR-IAA-09	M	The bottom navigation shall be hidden while the LOGIN view is active.
FR-IAA-10	S	The login form shall display a HIPAA compliance notice: "HIPAA-Compliant · End-to-end encrypted intake · No video infrastructure used".
FR-IAA-11	M	The form shall use novalidate to suppress native browser validation and rely solely on application-level logic.

3.2 Specialist Dashboard

The dashboard presents all available dental specialists in a card-grid layout. It is the primary discovery and navigation hub post-authentication.

Req ID	Pri	Requirement
FR-DSH-01	M	The system shall render a personalised header displaying "Welcome back, [FirstName]" and a patient avatar chip showing the first initial of the patient's name.
FR-DSH-02	M	The system shall render a card for every dentist record returned by DentalDB.getAllDentists() (minimum 6 in v1.0).
FR-DSH-03	M	Each dentist card shall display: name, specialty, credentials, years of experience, star rating (formatted to 2 decimal places), review count, avatar initials, clinic name, and specialty tags.
FR-DSH-04	M	Each dentist card shall apply the dentist's accentColor as a CSS custom property (--accent) for visual distinction.
FR-DSH-05	M	Each dentist card shall contain a "Select & Book Appointment" button that calls AppState.selectDentist(id) then transitions to the REGISTRATION view.
FR-DSH-06	S	Dentist cards shall render in a responsive grid layout adapting from single-column (mobile) to multi-column (desktop).

3.3 Clinical Registration & Booking Form

The registration view presents a structured multi-section form enabling the patient to supply clinical intake data and select an appointment slot.

Req ID	Pri	Requirement
FR-REG-01	M	The system shall redirect to the DASHBOARD view if the registration form is accessed without a selected dentist (selectedDentistId is null).
FR-REG-02	M	The form shall display a dentist profile banner showing the selected dentist's name, specialty, credentials, years of experience, clinic name, and accent-coloured avatar with initials.
FR-REG-03	M	Section 1 — Patient Information: shall contain pre-populated read-only or editable fields for Full Name and Mobile Phone, sourced from state.patient.
FR-REG-04	M	Section 2 — Reason for Visit: shall contain a <select> element populated with all categories returned by DentalDB.getSymptomCategories() (8 categories in v1.0).
FR-REG-05	M	Section 3 — Symptoms & Notes: shall contain a <textarea> with a 500-character hard limit; a live character counter shall update on every keystroke.
FR-REG-06	M	Section 4 — Appointment Slot: shall render the dentist's slot matrix, grouped by date (day label), with each time rendered as a clickable button.
FR-REG-07	M	Clicking a slot button shall call AppState.selectSlot(isoDate, time); the selected button shall receive the slot-btn--selected CSS class; previously selected slots shall be deselected.

FR-REG-08	M	On form submission, the system shall invoke AppState.submitBooking(formData, onStatusChange).
FR-REG-09	M	Validation rules for registration shall match: fullName (FR-IAA-02), phone (FR-IAA-03), symptomCategory (non-empty), symptomNotes (10–500 characters), and slot (a date and time must be selected).
FR-REG-10	M	On validation failure, inline field-level error messages shall render and a toast shall display: title "Form Incomplete", body "Please review the highlighted fields."
FR-REG-11	M	The submit button shall be disabled and show a loading state during async pipeline execution.
FR-REG-12	M	A "Back" control shall allow the patient to return to the DASHBOARD view without submitting.
FR-REG-13	S	The phone field on the registration form shall also apply real-time (XXX) XXX-XXXX auto-formatting.

3.4 Appointment Confirmation & Notification

Upon successful booking submission, the system executes a multi-stage asynchronous pipeline simulating carrier-grade SMS delivery and renders a confirmation view.

Req ID	Pri	Requirement
FR-CNF-01	M	The booking pipeline shall execute the following state transitions in sequence: validating → queued → sending → confirmed (or failed on error).
FR-CNF-02	M	Each pipeline stage shall have a randomised delay (jitter): validating 400–800 ms; queued 500–900 ms; sending 900–1600 ms.
FR-CNF-03	M	At the "queued" stage, a toast shall display: title "Intake Validated", body "Your request has been queued for confirmation."
FR-CNF-04	M	On confirmed status, the system shall generate a confirmation record containing: unique ID (MC-[base36 timestamp]), ISO timestamp, full patient snapshot, full dentist snapshot, appointment date/time, symptom category, symptom notes, and formatted SMS + push message content.
FR-CNF-05	M	The confirmation ID shall follow the format MC-XXXXXXXX using Date.now().toString(36).toUpperCase().
FR-CNF-06	M	The SMS message content shall follow the template: "MedConnect: Hi [FirstName], your appointment with [DentistName] is CONFIRMED for [Long Date] at [Time]. Location: [ClinicName]. Reply C to cancel."
FR-CNF-07	M	On confirmed status, a success toast shall display the push notification payload: title "Appointment Confirmed ✅"; body "[DentistName] · [Long Date] · [Time]".
FR-CNF-08	M	The system shall transition to the CONFIRMATION view immediately after the confirmed status fires.

FR-CNF-09	M	The confirmation view shall display: confirmation ID, patient full name, dentist name, appointment date (long format), appointment time, reason for visit, and clinic name.
FR-CNF-10	M	The confirmation view shall display an SMS preview bubble showing the phone number the SMS was sent to and the full SMS message text.
FR-CNF-11	M	The confirmation view shall contain a "Book Another Appointment" button that calls AppState.resetBookingFlow(), resetting selectedDentistId, selectedDate, selectedTime, and booking state, then transitions to the DASHBOARD view.

3.5 State-Driven Navigation

The bottom navigation bar provides primary wayfinding and enforces access control based on application state.

Req ID	Pri	Requirement
FR-NAV-01	M	The bottom navigation shall display three items: Dentists (🦷), Booking (📋), Confirmed (✅).
FR-NAV-02	M	The Booking navigation item shall be disabled (and non-interactive) when no dentist has been selected (selectedDentistId is null).
FR-NAV-03	M	The Confirmed navigation item shall be disabled when booking.status is not "confirmed".
FR-NAV-04	M	The active view's navigation item shall receive the nav-item--active CSS class.
FR-NAV-05	M	The bottom navigation shall be hidden entirely during the LOGIN view.
FR-NAV-06	M	Tapping/clicking an enabled navigation item shall call AppState.setView(view) with the corresponding view constant.

4. Data Requirements

4.1 Dentist Record Schema

Field	Type	Constraint	Description
id	String	Unique, kebab-case	Stable identifier (e.g., "dr-amelia-hart")
name	String	Non-empty	Full name with credential suffix
specialty	String	Non-empty	Primary dental specialty
credentials	String	Non-empty	Degree and institution
yearsExperience	Integer	> 0	Years of clinical practice
rating	Float	1.0 – 5.0	Aggregate patient rating
reviewCount	Integer	≥ 0	Total number of reviews
avatarInitials	String	2 characters	Initials for avatar display
accentColor	String	Valid CSS hex (#RRGGBB)	Card and avatar accent colour
bio	String	Non-empty	Short clinical biography
languages	String[]	≥ 1 element	Languages spoken
clinic.name	String	Non-empty	Clinic display name
clinic.address	String	Non-empty	Full clinic street address
tags	String[]	1–5 elements	Specialty quick-reference tags
slots	SlotDay[]	5 entries	Slot matrix for next 5 business days

4.2 Slot Matrix Schema

Field	Type	Constraint	Description
isoDate	String	ISO 8601 (YYYY-MM-DD)	Calendar date of the slot day
dayLabel	String	Non-empty	Human-readable label (e.g., "Mon, Jul 15")
slots	String[]	≥ 1 element	Available time strings (e.g., "09:15 AM")

4.3 Patient State Schema

Field	Type	Constraint	Description

identityVerified	Boolean	Default: false	Set to true after successful login
fullName	String	≥ 3 chars	Patient legal name
phone	String	10 numeric digits	Formatted US phone number
email	String	Optional, valid format	Patient email address

4.4 Confirmation Record Schema

Field	Type	Description
id	String	Unique confirmation token (MC-XXXXXXXX)
timestamp	String	ISO 8601 datetime of confirmation
patient	PatientState	Deep copy of patient state at time of booking
dentist	DentistRecord	Deep copy of selected dentist record
isoDate	String	Selected appointment date
time	String	Selected appointment time
symptomCategory	String	Selected reason for visit
symptomNotes	String	Patient-supplied symptom description
message.sms	String	Complete SMS text body
message.push.title	String	Push notification title
message.push.body	String	Push notification body

5. External Interface Requirements

5.1 User Interfaces

UI Element	Requirement
Colour Palette	Dark theme: primary background #0B1B22; surface cards elevated via border/shadow; no purely white backgrounds in primary surfaces.
Typography	Fraunces (display/headings), Inter (body/UI), IBM Plex Mono (confirmation IDs, code-like values).
Layout	Single scrollable column; max-width container centred on desktop; full-bleed on mobile.
Toast Notifications	Fixed overlay; top-right or bottom-centre; auto-dismiss after 4200 ms; visible CSS transition (toast--visible class).
Error States	Field-level inline text below input; input border/ring changes via input--error class; no modal dialogs for validation.
Loading States	Submit button disabled + btn--loading class; button label text updates to reflect pipeline stage.
Accessibility	aria-live="polite" on view-root and toast-host; nav uses <nav> with aria-label; all interactive elements keyboard-focusable.

5.2 Software Interfaces

Interface	Type	Description
DentalDB.getAllDentists()	Internal JS API	Returns array of all dentist records
DentalDB.getDentistById(id)	Internal JS API	Returns single dentist or null
DentalDB.getSymptomCategories()	Internal JS API	Returns array of symptom category strings
AppState.authenticate(obj)	Internal JS API	Sets patient state and transitions to DASHBOARD
AppState.selectDentist(id)	Internal JS API	Sets selectedDentistId; resets slot selection
AppState.selectSlot(date, time)	Internal JS API	Updates selectedDate and selectedTime
AppState.submitBooking(data, cb)	Internal JS API	Executes async pipeline; calls cb at each stage
AppState.resetBookingFlow()	Internal JS API	Clears booking state; transitions to DASHBOARD

AppState.subscribe(fn)	Internal JS API	Registers a render listener on every state emit
Google Fonts CDN	External HTTP	Delivers Fraunces, Inter, IBM Plex Mono typefaces

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Initial page load (all assets) shall complete in under 3 seconds on a standard broadband connection (50 Mbps).
NFR-02	Performance	View transitions shall complete within 100 ms of state mutation.
NFR-03	Reliability	The booking pipeline jitter range shall not cause the total confirmation cycle to exceed 4 seconds under normal execution.
NFR-04	Usability	All interactive elements shall have a minimum touch target size of 44×44 CSS pixels (WCAG 2.5.5).
NFR-05	Usability	Error messages shall be specific and actionable (no generic "invalid input" strings).
NFR-06	Accessibility	All dynamic content updates shall be announced to screen readers via aria-live="polite" regions.
NFR-07	Accessibility	Colour contrast ratio for all text elements shall meet WCAG 2.1 AA minimum (4.5:1 for normal text; 3:1 for large text).
NFR-08	Security	No patient data shall be transmitted to an external server in v1.0 (all state is in-memory, client-side only).
NFR-09	Security	The form shall not use the HTML autocomplete="off" suppression to avoid breaking browser autofill for accessibility tools.
NFR-10	Maintainability	Each JS module (data.js, state.js, app.js) shall remain independently replaceable without changes to the other modules.
NFR-11	Maintainability	Adding a new dentist record shall require only a single object append to the dentists array in data.js.
NFR-12	Portability	The system shall function correctly when served from any HTTP origin (including file://) on supported browsers.
NFR-13	Privacy	No analytics, tracking scripts, or third-party cookies shall be loaded by the application.

7. Constraints & Compliance

ID	Domain	Constraint / Compliance Statement
CON-01	Legal	The system is a non-certified prototype. It must not be used to collect, store, or transmit real patient health information without a full HIPAA compliance audit and Business Associate Agreement (BAA) with all hosting providers.
CON-02	Data	In v1.0, all patient data is volatile (stored in browser memory only). A page refresh permanently destroys all session data. The system must not imply data persistence to the user.
CON-03	Telephony	No SMS messages are sent to real phone numbers. The SMS pipeline is a UI simulation only. Any future version integrating live telephony must comply with TCPA (US) and relevant regional telecommunications regulations.
CON-04	Scheduling	Slot availability is computed deterministically from the client clock and a seed function. No real-time scheduling database is queried. Displayed slots do not represent actual dentist availability.
CON-05	Accessibility	The system must conform to WCAG 2.1 Level AA as a design target. Full conformance audit is required before clinical deployment.
CON-06	Licensing	The codebase is released under the MIT License. Any deployment using patient data must assess whether MIT licensing is compatible with their legal obligations.
CON-07	Scope	Telemedicine, video consultation, e-prescription, and payment processing are explicitly out of scope for v1.0.

Appendix A — Use Case Summary

UC ID	Use Case	Actor	Trigger	Success Outcome
UC-01	Authenticate Identity	Patient	Opens application URL	Patient lands on Dashboard; welcome toast displayed
UC-02	Browse Specialists	Patient	Views Dashboard	Patient sees all dentist cards with profile data
UC-03	Select Specialist	Patient	Taps "Select & Book"	Registration form opens with selected dentist banner
UC-04	Complete Registration	Patient	Fills all form sections	All fields pass validation; submit button enabled
UC-05	Select Appointment Slot	Patient	Taps a slot button	Slot highlighted; date/time stored in AppState
UC-06	Submit Booking	Patient	Taps "Confirm Appointment"	Pipeline completes; confirmation view displayed
UC-07	View Confirmation	Patient	Pipeline reaches "confirmed"	Confirmation ID, SMS preview, and details shown
UC-08	Book Another Appointment	Patient	Taps "Book Another"	State reset; patient returned to Dashboard

Appendix B — Application State Machine

The following table represents the formal state transition model for the MedConnect application view router and booking pipeline.

B.1 View State Transitions

Current State	Event / Guard	Next State
LOGIN	authenticate() called; validation passes	DASHBOARD
DASHBOARD	selectDentist(id) + setView(REGISTRATION)	REGISTRATION
DASHBOARD	nav item "Confirmed" tapped [booking = confirmed]	CONFIRMATION
REGISTRATION	back button tapped	DASHBOARD
REGISTRATION	submitBooking() → confirmed	CONFIRMATION
REGISTRATION	nav item "Dentists" tapped	DASHBOARD
CONFIRMATION	resetBookingFlow()	DASHBOARD
CONFIRMATION	nav item "Dentists" tapped	DASHBOARD
ANY	nav item disabled (guard fails)	No transition

B.2 Booking Pipeline State Transitions

Pipeline State	Trigger	Next State	Delay
idle	submitBooking() called	validating	Immediate
validating	Simulated backend check completes	queued	400–800 ms
queued	SMS gateway handoff simulated	sending	500–900 ms
sending	Carrier delivery simulated	confirmed	900–1600 ms
validating	Validation errors detected	failed	Immediate

Appendix C — Data Dictionary

Term	Source Module	Type	Description
dentists	data.js	Array	Private array of 6 DentistRecord objects; the single source of truth for all specialist data
symptomCategories	data.js	String[]	Ordered list of 8 visit-reason categories surfaced as dropdown options
generateSlotMatrix(seed)	data.js	Function	Deterministic pseudo-random slot generator; produces 5 business-day slot arrays; suppresses Sundays
state	state.js	Object	Singleton canonical application state: currentView, patient, selectedDentistId, selectedDate, selectedTime, booking, notificationQueue, listeners
VIEWS	state.js	Frozen Object	Enum constants: LOGIN | DASHBOARD | REGISTRATION | CONFIRMATION
ValidationRules	state.js	Object	Namespace of pure validation functions (fullName, phone, email, symptomCategory, symptomNotes, slotSelected); each returns null (pass) or error string (fail)
notificationQueue	state.js	Array	Append-only log of all ConfirmationRecords generated in the session
booking.status	state.js	Enum String	idle | validating | queued | sending | confirmed | failed
formatPhone(value)	state.js	Function	Strips non-digits, caps at 10, formats as (XXX) XXX-XXXX
jitter(min, max)	state.js	Function	Returns random integer in [min, max] for pipeline delay simulation
buildConfirmationMessage()	state.js	Function	Constructs SMS text and push notification payload from booking data
root	app.js	DOM Element	The #view-root section; all view HTML is injected here on each render
bottomNav	app.js	DOM Element	The #bottom-nav element; re-rendered on every state change
toastHost	app.js	DOM Element	The #toast-host div; toasts are appended and auto-removed here
showToast(opts)	app.js	Function	Creates, animates, and auto-dismisses a toast element after 4200 ms

— End of Document —